

AIMLCZG546 – Software Engineering for Machine Learning

Assignment I – Project Report

Loan Default / Credit-Risk Prediction

Group No: 40

Group Members and Contribution

Sl.	BITS ID	Name	Contribution (Qualitative)	% (of 100)
1	<fill>	<fill>	<fill>	<fill>
2	<fill>	<fill>	<fill>	<fill>
3	<fill>	<fill>	<fill>	<fill>
4	<fill>	<fill>	<fill>	<fill>

ACTION REQUIRED: fill in the group members table above (BITS IDs, names, and each member's qualitative and percentage contribution) before submitting.

Notes on this report

- Several dataset column codes (e.g. loan_type, loan_purpose, Credit_Worthiness) have no official data dictionary; their meanings are reasonable inferences, not confirmed facts.
- The target column Status is treated as 1 = default, 0 = non-default. This is the conventional reading and is consistent with the data, but it is an inference and should be verified.
- The GR4ML notation follows the framework attributed to Nalchigar and Yu (University of Toronto). Verify the exact symbols against your course slides and cite your course reference; no specific paper is cited here to avoid an unverified citation.
- Reported metrics are the output of one specific configuration (fixed preprocessing, random seed 42, single train/test split) and are not definitive performance figures.
- Screenshots: the program outputs shown are the actual outputs captured during testing. Replace the marked placeholders with screenshots from your own run, as required by the assignment.

Contents

1. Introduction and Domain.....	3
2. Objective 1 – Requirements Formulation	3
2.1 Problem Statement.....	3
2.2 Functional Requirements	3
2.3 Non-Functional (Quality) Requirements	3
2.4 Measurable Goals	3
Business goals	3
ML / analytics goals.....	4
2.5 GR4ML Models.....	4
2.5.1 Business View.....	4
2.5.2 Analytics Design View	4
2.5.3 Data Preparation View	5
2.6 Top Three Quality Requirements and Justification.....	5
3. Data Understanding and Preparation	5
3.1 Target and Missing Values	5
3.2 Target Leakage (Important)	6
4. Objective 2 – System Architecture.....	6
4.1 Architecture Diagram (ML and non-ML components).....	6
4.2 Selected Architectural Patterns	6
4.3 Implementation	6
4.3.1 Technologies and Tools.....	6
4.3.2 Pattern 1 – Pipe-and-Filter (loan_default_pipeline.py)	7
4.3.3 Model Selection Results.....	7
4.3.4 Pattern 2 – Model-as-a-Service (loan_default_service.py)	7
4.3.5 Service Output	8
4.3.6 How to Reproduce	8
5. Limitations and Honest Caveats.....	8
Appendix A – Full Code: loan_default_pipeline.py.....	9
Appendix B – Full Code: loan_default_service.py.....	13

1. Introduction and Domain

This project applies software engineering principles to a machine learning application in the finance domain: predicting whether a loan applicant is likely to default. A lending institution earns revenue by issuing loans but carries credit risk when borrowers default. Manual and static rule-based underwriting is slow, inconsistent, and can miss complex patterns, motivating an ML-based credit-risk decision-support system.

Dataset used: **yasserh/loan-default-dataset** from Kaggle, loaded with pandas. Measured on the file: 148,670 rows and 34 columns. The target column is **Status**.

2. Objective 1 – Requirements Formulation

2.1 Problem Statement

Build an ML-based credit-risk decision-support application that, at loan-application time, estimates each applicant's probability of default from their income, credit history, loan characteristics and property details, so the institution can approve, decline, or route an application to manual review — reducing default-related losses while keeping decisions consistent, explainable, and fair. This is an application-time (origination) decision, not real-time per-transaction scoring.

2.2 Functional Requirements

- Ingest a loan-application record (income, loan amount, credit score, LTV, DTI ratio, property value, term, etc.).
- Preprocess: handle missing values, encode categorical fields, and scale numeric features.
- Score each application to produce a default-probability output.
- Apply a decision policy: approve / decline / refer to manual review based on the score.
- Produce a per-decision explanation of the main contributing factors.
- Log decisions and repayment outcomes, and support periodic retraining.

2.3 Non-Functional (Quality) Requirements

- Predictive performance under class imbalance.
- Interpretability / explainability of decisions.
- Fairness / non-discrimination (dataset contains sensitive attributes).
- Robustness to data-quality issues (missing values, multicollinearity).
- Auditability / traceability of decisions.
- Security and privacy of financial personal data.

2.4 Measurable Goals

Target values below are illustrative placeholders to be set and justified by the team; they are not benchmarks.

Business goals

- Reduce default-related losses on the approved portfolio by a target percentage over a defined period.
- Keep the false-decline rate on creditworthy applicants below a set ceiling.
- Return an automated decision within a target time (e.g. a few seconds).

ML / analytics goals

- Achieve a target recall on the default class (missed defaults are costly).
- Maintain precision above a floor to keep review volumes manageable.
- Report ROC-AUC and PR-AUC as headline metrics instead of plain accuracy, because the target is imbalanced.
- Aim for well-calibrated default probabilities.

2.5 GR4ML Models

The application is modelled with the three GR4ML views. The notation follows the framework attributed to Nalchigar and Yu; verify the exact symbols against your course slides.

2.5.1 Business View

Connects enterprise strategy to the analytics. Core elements mapped to this application:

- Actor: Loan Officers / Credit Underwriters.
- Business Goal (ellipse): Process loan applications — desired by the actor.
- Indicator (traffic light): Default Loss Rate — evaluates the business goal.
- Decision Goal (D-ellipse): Decision on loan application (approve / decline / refer) — AND-refines the business goal.
- Question Goal (Q-ellipse): What is the default risk of the current applicant? — AND-refines the decision goal.
- Insight (rectangle): Loan Default Predictive Model, with attributes type / input / output / usage-frequency / update-frequency / learning-period — answers the question and generates the insight.

INSERT DIAGRAM: GR4ML Business View (paste the diagram rendered during the session, or your redrawn version in the exact course notation).

2.5.2 Analytics Design View

- Analytics Goal (ellipse): Classification of an applicant/loan profile as default / non-default (a Prediction Goal) — is required for the business insight.
- Algorithms (hexagons, each 'performs' the goal): Logistic Regression, Random Forest, Gradient Boosting.
- Softgoals (clouds): Interpretability, Fairness, Tolerance to missing values, Robustness to class imbalance.
- Indicators (traffic light): Recall / PR-AUC, Precision — evaluate the goal.
- Influence links (+/-): Logistic Regression + Interpretability; Gradient Boosting - Interpretability, + predictive performance; excluding Gender + Fairness.

INSERT DIAGRAM: GR4ML Analytics Design View (to be drawn in the course notation from the elements listed above).

2.5.3 Data Preparation View

- Entity (rectangle, PK + attributes): Loan Application — PK: ID; income, Credit_Score, age, loan_amount, term, loan_type, loan_purpose, occupancy_type, property_value, LTV, dtir1, ...; Status (label feature).
- Operators (rectangles, joined by data flow): Filter -> Impute missing values -> Encode categoricals -> Normalize/Scale -> Balance classes -> outputs prepared features.
- Note: drop [ID, Gender] and the leakage columns; impute; one-hot encode. (The single flat CSV means a Join operator is optional.)

INSERT DIAGRAM: GR4ML Data Preparation View (to be drawn in the course notation from the elements listed above).

2.6 Top Three Quality Requirements and Justification

Which three are 'top' is a design judgment. The trio below is defensible; the third can be swapped for data-quality robustness depending on the team's emphasis.

1. Interpretability / Explainability. Lending decisions typically must be justifiable to applicants, auditors, and regulators, so a black-box output is hard to defend, contest, or govern. This drives the choice between inherently interpretable models and post-hoc explanations.

2. Fairness / Non-discrimination. The dataset contains Gender and age (confirmed in the column list), which are sensitive attributes in most lending contexts. A model using them, or their proxies, can produce disparate outcomes. This becomes a testable requirement: exclude Gender and evaluate bias across groups. (The applicable legal standard is jurisdiction-dependent and should be verified.)

3. Predictive performance under class imbalance. The target is imbalanced — measured at 75.4% non-default vs 24.6% default — so accuracy is misleading (predicting 'no default' for everyone already scores ~75%). The meaningful requirement is strong performance on the minority class, measured by recall and PR-AUC.

Alternative third requirement: robustness to data-quality issues, justified by the dataset's documented missing values and multicollinearity and by the leakage found during analysis (Section 3).

3. Data Understanding and Preparation

Findings below were computed directly on the uploaded dataset and are stated as verified facts; interpretations of why a pattern occurs are marked as such.

3.1 Target and Missing Values

- Target balance (verified): Status = 0 in 75.4% of rows, Status = 1 in 24.6% (112,031 vs 36,639). The imbalance is moderate, not extreme.

- Missing values are substantial and vary by column (verified): e.g. Upfront_charges ~26.7%, Interest_rate_spread ~24.6%, rate_of_interest ~24.5%, dtir1 ~16.2%, property_value / LTV ~10.2%.

3.2 Target Leakage (Important)

Two separate target leaks were identified by inspecting columns against the target. The statistical patterns are verified facts; describing them as artifacts is an inference.

- Missingness leakage: Interest_rate_spread is missing for exactly the defaulted rows — $P(\text{default} \mid \text{missing}) = 1.000$ and $P(\text{default} \mid \text{present}) = 0.000$. rate_of_interest is almost identical; Upfront_charges is ~92% aligned.
- Category leakage: within credit_type, every 'EQUI' row (15,298 rows) is a default (100% default rate), while the other bureaus sit near the ~16% baseline.

Mitigation applied: the pipeline drops Interest_rate_spread, rate_of_interest, Upfront_charges and credit_type, plus ID (identifier) and Gender (fairness). Removing credit_type is a safe choice that also discards mild signal from the other bureaus.

4. Objective 2 – System Architecture

4.1 Architecture Diagram (ML and non-ML components)

The online decision path (request-time flow) contains both ML and non-ML components:

- Non-ML: Loan Officer Portal (UI), Application API (validation), Decision Engine (approve/decline/refer thresholds), Manual Review Queue, Application & Outcomes database.
- ML: Feature Preprocessing, Model Inference Service, Explainability Module.
- Offline side: Data Preparation pipeline, Model Training, Model Evaluation, Model Registry (storage), and Monitoring / drift detection that triggers retraining.

INSERT DIAGRAM: System architecture (paste the serving-path diagram rendered during the session, and, if drawn, the offline training/monitoring companion diagram).

4.2 Selected Architectural Patterns

Pattern 1 – Pipe-and-Filter: the data-processing and training workflow is a chain of independent transformation stages (filters) connected by pipes.

Pattern 2 – Microservice / Model-as-a-Service: the trained model is packaged behind an HTTP service that any client can call, decoupling the model from the rest of the system.

The two connect: the Pipe-and-Filter pipeline trains and serializes the model that the Microservice loads and serves. Retraining only swaps the saved model file; no client code changes.

4.3 Implementation

4.3.1 Technologies and Tools

Python 3; pandas; scikit-learn (Pipeline, ColumnTransformer, SimpleImputer, StandardScaler, OneHotEncoder, LogisticRegression, RandomForestClassifier, HistGradientBoostingClassifier); joblib for

model persistence; Flask for the HTTP service; requests/curl as client. Tested with scikit-learn 1.8.0, Flask 3.1.3, pandas 3.0.2 — record the versions you actually run.

4.3.2 Pattern 1 – Pipe-and-Filter (loan_default_pipeline.py)

Implemented as a single scikit-learn Pipeline built from a ColumnTransformer. Numeric filter chain: median imputation then standard scaling. Categorical filter chain: most-frequent imputation then one-hot encoding. The full sequence is: load -> drop non-predictive/leaky columns -> impute -> encode -> scale -> model. Three classifiers are compared on a stratified 25% hold-out and the best is selected by PR-AUC.

4.3.3 Model Selection Results

Results from one run (seed 42, single split); config-dependent, not definitive:

Model	Accuracy	Recall	PR-AUC	ROC-AUC
Logistic Regression	0.668	0.644	0.501	0.724
Random Forest	0.889	0.590	0.831	0.889
Hist Gradient Boosting (selected)	0.874	0.728	0.842	0.895

Captured program output:

```
[LOAD] Reading loan_default_dataset.csv
[LOAD] Shape: 148,670 rows x 34 columns
[LOAD] Target balance -> 0 (non-default): 75.4% | 1 (default): 24.6%
[PREP] Dropping columns: ['ID', 'Gender', 'Interest_rate_spread',
    'rate_of_interest', 'Upfront_charges', 'credit_type']
[PREP] 8 numeric, 19 categorical features
[SPLIT] train 111,502 | test 37,168 (stratified)
[TRAIN] Logistic Regression ...
[TRAIN] Random Forest ...
[TRAIN] Hist Gradient Boosting ...
=====
MODEL COMPARISON (test set)
=====
              accuracy precision recall    f1  roc_auc  pr_auc
Logistic Regression    0.6684    0.3943  0.6443 0.4892    0.7235 0.5008
Random Forest          0.8887    0.9345  0.5898 0.7232    0.8889 0.8309
Hist Gradient Boosting 0.8737    0.7517  0.7281 0.7397    0.8945 0.8418
[SELECT] Best model by PR-AUC: Hist Gradient Boosting
Confusion matrix for Hist Gradient Boosting:
      pred:0  pred:1
actual 0    25805   2203
actual 1     2491   6669
[SAVE] Selected model 'Hist Gradient Boosting' saved to loan_default_model.joblib
```

INSERT SCREENSHOT: terminal showing the output of running loan_default_pipeline.py on your machine.

4.3.4 Pattern 2 – Model-as-a-Service (loan_default_service.py)

A Flask microservice loads the saved model and exposes two endpoints: GET / (health check reporting the number of expected features) and POST /predict (accepts one application as a JSON object or many as a JSON list, and returns a default probability plus a decision). A non-ML decision policy applies approve / refer / decline thresholds on top of the model score. Requests are reindexed to the exact columns the pipeline expects, so omitted fields are imputed rather than causing errors.

4.3.5 Service Output

Captured responses (actual, from testing):

```
=== GET / (health check) ===
{ "status": "ok", "service": "loan-default-scoring",
  "model_expects_n_features": 27, "decline_at": 0.7, "refer_at": 0.4 }

=== POST /predict : two real applications from the dataset ===
[ { "decision": "refer_to_manual_review", "default_probability": 0.514 },
  { "decision": "approve", "default_probability": 0.0879 } ]

=== POST /predict : a single risky application ===
{ "decision": "decline", "default_probability": 0.9637 }
```

INSERT SCREENSHOT: terminal/Postman showing the health check and a /predict call on your machine.

4.3.6 How to Reproduce

```
# 1. Pipe-and-Filter: train + save the model
python3 loan_default_pipeline.py loan_default_dataset.csv

# 2. Model-as-a-Service: start the microservice
python3 loan_default_service.py

# 3. Call the service (in another terminal)
curl http://localhost:5000/
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" -d @sample_request.json
```

5. Limitations and Honest Caveats

- Status = 1 is assumed to mean 'default'; the dataset has no official data dictionary, so this is an inference.
- The meanings of coded categorical values (cf/ncf, l1/l2, p1–p4, etc.) are inferences, not documented facts.
- The decision thresholds (0.70 decline / 0.40 refer) are illustrative placeholders, not values validated on data.
- Reported metrics depend on the preprocessing choices, the random seed, and a single train/test split; cross-validation would give a more defensible estimate.
- Two target leaks were found and removed, but the data cannot be guaranteed leakage-free; the tree models' scores may still be optimistic.
- Flask's built-in server is for development/demonstration only; a production deployment would use, e.g., FastAPI with gunicorn/uvicorn.
- The group members table and all screenshots must be completed by the team before submission.

Appendix A – Full Code: loan_default_pipeline.py

```
"""
loan_default_pipeline.py
=====
Loan Default Prediction - end-to-end ML pipeline.

Assignment: AIMLCZG546 - Software Engineering for Machine Learning
Group: <fill in your group no and member names>

Architectural pattern demonstrated: PIPE-AND-FILTER.
Each stage is a "filter" that transforms the data and passes it to the next:
    raw data -> impute missing -> encode categoricals -> scale numerics -> model
This is implemented with scikit-learn's ColumnTransformer + Pipeline, so the
whole flow is a single composable object.

IMPORTANT NOTES / ASSUMPTIONS (verify these for your own submission):
* Target column `Status`: this code assumes 1 = default, 0 = non-default.
  That is the conventional reading and is consistent with the data, but it is
  an INFERENCE - the dataset ships with no official data dictionary.
* Leakage handling: `Interest_rate_spread`, `rate_of_interest`, and
  `Upfront_charges` are DROPPED. On this dataset their missingness aligns
  almost perfectly with the target (e.g. Interest_rate_spread is missing for
  exactly the defaulted rows), so keeping them would leak the answer.
* `Gender` is dropped as a fairness / non-discrimination design choice.
* `ID` is dropped (identifier, no predictive signal).
* Reported metrics depend on these preprocessing choices, the fixed random
  seed, and the assumed target direction. They are not "the" answer - they
  are what THIS configuration produces.
"""

import sys
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
)

# ----- #
# Configuration
# ----- #
TARGET = "Status"
ID_AND_SENSITIVE = ["ID", "Gender"]          # identifier + protected attribute

# Two kinds of target leakage were found by inspecting the data:
# (a) missingness leakage - these columns are missing almost exactly for the
#     defaulted rows (e.g. Interest_rate_spread is missing for 100% of defaults).
# (b) category leakage - within `credit_type`, the value 'EQUI' has a 100%
#     default rate (all 15,298 EQUI rows default), so the column partly encodes
#     the target. Dropping the whole column is the safe choice; the remaining
#     bureaus carry only mild signal.
# NOTE: (b) being an "artifact" rather than a genuine predictor is an inference;
# the 100% default rate itself is a verified fact from the data.
LEAKAGE_COLS = [
    "Interest_rate_spread", "rate_of_interest", "Upfront_charges", # (a)
    "credit_type",          # (b)
]
RANDOM_STATE = 42
TEST_SIZE = 0.25

# ----- #
# 1. DATA LOADING
# ----- #
def load_data(path):
```

```

print(f"[LOAD] Reading {path}")
df = pd.read_csv(path)
print(f"[LOAD] Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
balance = df[TARGET].value_counts(normalize=True).sort_index()
print(f"[LOAD] Target balance -> "
      f"0 (non-default): {balance.get(0, 0):.1%} | "
      f"1 (default): {balance.get(1, 0):.1%}")
return df

# ----- #
# 2. DATA PROCESSING (the Pipe-and-Filter preprocessing definition)
# ----- #
def split_features_target(df):
    drop = [c for c in (ID_AND_SENSITIVE + LEAKAGE_COLS) if c in df.columns]
    print(f"[PREP] Dropping columns: {drop}")
    df = df.drop(columns=drop)

    y = df[TARGET].astype(int)
    X = df.drop(columns=[TARGET])

    num_cols = X.select_dtypes(include=np.number).columns.tolist()
    cat_cols = X.select_dtypes(exclude=np.number).columns.tolist()
    print(f"[PREP] {len(num_cols)} numeric, {len(cat_cols)} categorical features")
    return X, y, num_cols, cat_cols

def build_preprocessor(num_cols, cat_cols):
    # Numeric filter chain: impute median -> standardize
    numeric_filter = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ])
    # Categorical filter chain: impute most-frequent -> one-hot encode
    categorical_filter = Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(handle_unknown="ignore")),
    ])
    return ColumnTransformer([
        ("num", numeric_filter, num_cols),
        ("cat", categorical_filter, cat_cols),
    ])

# ----- #
# 3. MODEL SELECTION
# ----- #
def candidate_models():
    # class_weight balanced addresses the ~75/25 imbalance.
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE),
        "Random Forest": RandomForestClassifier(
            n_estimators=200, class_weight="balanced_subsample",
            n_jobs=-1, random_state=RANDOM_STATE),
        "Hist Gradient Boosting": HistGradientBoostingClassifier(
            class_weight="balanced", random_state=RANDOM_STATE),
    }

def evaluate(name, model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]
    return {
        "model": name,
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "pr_auc": average_precision_score(y_test, y_proba),
    }

```

```

def select_models(X, y, num_cols, cat_cols):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE)
    print(f"[SPLIT] train {X_train.shape[0]:,} | test {X_test.shape[0]:,} "
          f"(stratified)")

    pre = build_preprocessor(num_cols, cat_cols)
    results, fitted = [], {}

    for name, clf in candidate_models().items():
        print(f"[TRAIN] {name} ...")
        pipe = Pipeline([("preprocess", pre), ("model", clf)])
        pipe.fit(X_train, y_train)
        results.append(evaluate(name, pipe, X_test, y_test))
        fitted[name] = pipe

    results_df = pd.DataFrame(results).set_index("model")
    return results_df, fitted, (X_test, y_test)

# ----- #
# 4. OUTPUT
# ----- #
def report(results_df, fitted, test_data):
    X_test, y_test = test_data
    pd.set_option("display.width", 120)
    pd.set_option("display.float_format", lambda v: f"{v:.4f}")

    print("\n" + "=" * 70)
    print("MODEL COMPARISON (test set)")
    print("=" * 70)
    print(results_df.to_string())

    # Selection rule: prioritise PR-AUC, the appropriate metric under imbalance.
    best_name = results_df["pr_auc"].idxmax()
    print(f"\n[SELECT] Best model by PR-AUC: {best_name}")
    print(f"      (PR-AUC / recall are prioritised over accuracy because the "
          "positive\nclass is the minority - accuracy alone is misleading.)")

    best = fitted[best_name]
    y_pred = best.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    print(f"\nConfusion matrix for {best_name}:")
    print("      pred:0    pred:1")
    print(f"actual 0    {cm[0, 0]:>8d} {cm[0, 1]:>8d}")
    print(f"actual 1    {cm[1, 0]:>8d} {cm[1, 1]:>8d}")

    # Interpretability view (quality requirement). The best model by PR-AUC may
    # be a boosting model that exposes no importances, so we always show the
    # interpretable reference (Logistic Regression coefficients) plus Random
    # Forest importances where available.
    if "Logistic Regression" in fitted:
        print_top_features(fitted["Logistic Regression"], "Logistic Regression")
    if "Random Forest" in fitted:
        print_top_features(fitted["Random Forest"], "Random Forest")
    return best_name, best

def print_top_features(pipe, name, k=12):
    """Show the most influential features (interpretability quality req)."""
    try:
        feat_names = pipe.named_steps["preprocess"].get_feature_names_out()
    except Exception as e:
        print(f"\n[FEATURES] Could not extract feature names: {e}")
        return

    model = pipe.named_steps["model"]
    if hasattr(model, "feature_importances_"):
        vals, label = model.feature_importances_, "importance"
    elif hasattr(model, "coef_"):
        vals, label = np.abs(model.coef_[0]), "|coefficient|"
    else:
        print(f"\n[FEATURES] Model exposes no importances/coefficients.")
        return

```

```

order = np.argsort(vals)[::-1][:k]
print(f"\nTop {k} features by {label} ({name}):")
for i in order:
    print(f"    {feat_names[i]:40s} {vals[i]:.4f}")

# ----- #
# Orchestration
# ----- #
def main(path):
    df = load_data(path)
    X, y, num_cols, cat_cols = split_features_target(df)
    results_df, fitted, test_data = select_models(X, y, num_cols, cat_cols)
    best_name, best = report(results_df, fitted, test_data)

    import joblib
    out = "loan_default_model.joblib"
    joblib.dump(best, out)
    print(f"\n[SAVE] Selected model '{best_name}' saved to {out}")

if __name__ == "__main__":
    data_path = sys.argv[1] if len(sys.argv) > 1 else "loan_default_dataset.csv"
    main(data_path)

```

Appendix B – Full Code: loan_default_service.py

```
"""
loan_default_service.py
=====
Model-as-a-Service (Microservice) for loan default prediction.

Assignment: AIMLCZG546 - Software Engineering for Machine Learning
Group: <fill in your group no and member names>

Architectural pattern demonstrated: MICROSERVICE / MODEL-AS-A-SERVICE.
The trained model (produced by loan_default_pipeline.py, which implements the
Pipe-and-Filter pattern) is packaged behind a small HTTP service. Any client
- a loan-officer portal, a batch job, the decision engine - scores an
application by POSTing JSON to /predict, without knowing anything about the
model internals. The model can be retrained and the .joblib swapped without
changing any client.

ENDPOINTS
GET / -> health check + how many features the model expects
POST /predict -> accepts one application (JSON object) or many (JSON list),
returns default probability + a decision.

ASSUMPTIONS / CAVEATS (kept explicit on purpose):
* "default_probability" is P(Status = 1). This rests on the inference that
  Status = 1 means "default"; the dataset ships no data dictionary, so verify.
* The DECLINE_AT / REFER_AT thresholds below are ILLUSTRATIVE business policy,
  not values tuned or validated on data. Set them from your own cost analysis.
* Model quality depends on loan_default_pipeline.py's preprocessing choices
  (leakage columns dropped, etc.). See that file's notes.
* This uses Flask's built-in server, which is for development/demo only, not
  production. (FastAPI + a production server such as gunicorn/uvicorn would
  be a common alternative.)
"""

import joblib
import pandas as pd
from flask import Flask, request, jsonify

MODEL_PATH = "loan_default_model.joblib"

# Decision policy (non-ML business rules applied on top of the model score).
# These are placeholders - replace with thresholds your team can justify.
DECLINE_AT = 0.70 # probability at/above which the application is declined
REFER_AT = 0.40 # probability at/above which it goes to manual review

app = Flask(__name__)

model = joblib.load(MODEL_PATH)
# The fitted preprocessor knows exactly which input columns it expects.
EXPECTED_COLUMNS = list(model.named_steps["preprocess"].feature_names_in_)

def decide(prob):
    if prob >= DECLINE_AT:
        return "decline"
    if prob >= REFER_AT:
        return "refer_to_manual_review"
    return "approve"

@app.route("/", methods=["GET"])
def health():
    return jsonify({
        "status": "ok",
        "service": "loan-default-scoring",
        "model_expects_n_features": len(EXPECTED_COLUMNS),
        "decline_at": DECLINE_AT,
        "refer_at": REFER_AT,
    })
```

```

@app.route("/predict", methods=["POST"])
def predict():
    payload = request.get_json(force=True, silent=True)
    if payload is None:
        return jsonify({"error": "Body must be JSON (an object or a list of objects)."}), 400

    records = payload if isinstance(payload, list) else [payload]

    # Build a DataFrame with exactly the columns the pipeline expects.
    # Any feature the caller omits becomes NaN and is handled by the pipeline's
    # imputers; unknown categories are handled by OneHotEncoder(handle_unknown).
    df = pd.DataFrame(records).reindex(columns=EXPECTED_COLUMNS)

    probs = model.predict_proba(df)[: , 1]
    results = [
        {"default_probability": round(float(p), 4), "decision": decide(p)}
        for p in probs
    ]
    return jsonify(results[0] if len(results) == 1 else results)

if __name__ == "__main__":
    # host=0.0.0.0 so it is reachable locally for the demo.
    app.run(host="0.0.0.0", port=5000)

```